SAFI

SOW-04
Root Cause Analyzer for OpenAPI API Testing Harness

Six-Hour Product SOW | Distribution Copy

CENTRAL ACCEPTANCE PRINCIPLE

Each developer is accepted by cross-testing the other two developers' API Gateways. Self-testing is useful during development but does not count as final acceptance. All three Gateways, Swagger UI documentation, the automated Harness and the Root Cause Analyzer must share one protected canonical OpenAPI 3.1.2 contract.

Purpose. Build a reusable Root Cause Analyzer (RCA) that explains failed OpenAPI-driven API tests, produces a structured triage.json, and interoperates across independently implemented Gateways without Gateway-specific adapters.

Six-hour rule. One smart junior developer/fresher, using AI aggressively, must understand, build, integrate, test, fix, commit and deliver this Product within six working hours maximum.

1. Finished Product / Goal

At Hour 6, deliver a reusable Root Cause Analyzer that begins after an OpenAPI-driven API test fails and produces a machine-readable diagnosis containing:

- what failed and the earliest supported failing observation;
- the primary root-cause classification;
- the evidence supporting that classification;
- the applicable OpenAPI operation/contract reference;
- the suspected component and confidence;
- the smallest next investigation or repair area; and
- an explicit UNKNOWN or SPEC_AMBIGUITY result when evidence does not justify a stronger conclusion.

BOUNDARY

The RCA diagnoses. It does NOT modify application code, tests, the OpenAPI contract, expected outputs or the database. Automatic repair belongs to a later SOW.

2. Starting Point / Preconditions

- Three independently implemented API Gateways from the preceding SOW: Gateway A, Gateway B and Gateway C.
- One owner-supplied, protected canonical OpenAPI contract shared by all three Gateways.
- Existing OpenAPI API Testing Harness and its test-result artifacts.
- Existing Python runtime/repository conventions from the Harness.
- Company Git and AWS delivery environment.
- One approved LLM/API for runtime diagnosis, plus ordinary AI coding tools for development.

Reuse existing validators, request/response capture, logging and test-runner components. Do not replace working Harness infrastructure.

3. Mandatory Interoperability Contract

THIS IS A PASS/FAIL REQUIREMENT

If Developer A cannot test Gateway B and Gateway C directly through the common OpenAPI contract, Developer A does not pass. The same rule applies to B and C. No Gateway-specific adapter, private schema or handwritten endpoint mapping is permitted.

3.1 One Protected Canonical OpenAPI Description

The SOW owner supplies one canonical contract. It is immutable to the developer during acceptance:

```
contracts/openapi.yaml
openapi: 3.1.2
```

Proprietary and Copyright 2026 B. Gopinath

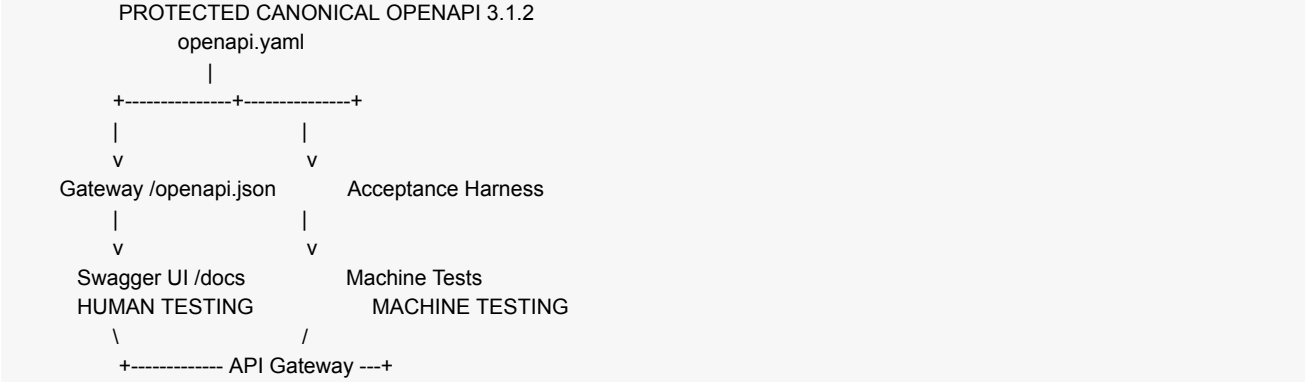
- The same canonical file is distributed to Developers A, B and C.
- Each Gateway must implement that contract; a developer may not weaken or rewrite it to match their implementation.
- Use a relative server URL or omit environment-specific server data so the contract remains common across deployments.
- The protected acceptance Harness validates each deployed Gateway against this owner-supplied contract.

3.2 Required Runtime Interface on Every Gateway

Path / Artifact	Mandatory Meaning
GET /openapi.json	Machine-readable OpenAPI 3.1.2 description of the deployed Gateway. It must be semantically identical to the protected canonical contract.
GET /docs	Swagger UI. It must render the Gateway's /openapi.json and expose Swagger “Try it out” / Execute for human testing.
application/json	Required request/response representation for acceptance-relevant body payloads unless the protected contract explicitly says otherwise.

“Semantically identical” means the parsed OpenAPI document describes the same paths, methods, operationIds, parameters, request schemas, response schemas and status codes. Formatting, YAML-versus-JSON serialization and key order do not matter. A protected test performs this comparison.

3.3 One Contract, Two Testing Interfaces



Swagger UI is the standard Safi human API documentation/testing interface. Do not substitute ReDoc, Scalar or a custom documentation page for acceptance.

3.4 Minimum Contract Discipline

- Every acceptance-relevant operation has a unique, stable operationId.
- Request parameters and request bodies are defined by OpenAPI schemas, not prose alone.
- Responses use explicit HTTP status codes and explicit response schemas where bodies are returned.
- The Harness identifies operations by HTTP method + path and records operationId in evidence.
- There is no separate “test schema,” “Swagger schema,” private endpoint map or second source of truth.
- The human-visible Swagger documentation and machine Harness testing describe the same API.

4. Final Acceptance Is Cross-Testing

Each developer may use their own Gateway while building. Final acceptance, however, is based on testing the other two independently built Gateways.

RCA / Harness Under Acceptance	Must Test	Does Not Count as Final Acceptance
Developer A	Gateway B + Gateway C	Gateway A only
Developer B	Gateway A + Gateway C	Gateway B only
Developer C	Gateway A + Gateway B	Gateway C only

4.1 Cross-Test Procedure

1. Start with a clean checkout of the developer's RCA/Harness work.
2. Supply only the target Gateway base URL plus the protected canonical openapi.yaml.
3. Fetch target /openapi.json and verify semantic equivalence to the protected contract.
4. Open target /docs and manually execute representative operations through Swagger UI.
5. Run the automated Harness against the target Gateway using the protected contract.
6. For every failed API test, invoke the RCA and produce triage.json.
7. Repeat against the second independent Gateway with no source-code changes and no Gateway-specific adapter.

Proprietary and Copyright 2026 B. Gopinath

8. Record PASS/FAIL evidence for both Gateways.

NO SPECIAL KNOWLEDGE

A fresher who did not build Gateway B must be able to understand and exercise B from /docs, and the automated Harness must be able to test B from the protected OpenAPI contract plus its base URL. Extra implementation-specific instructions are a failure of the interface.

5. RCA Product Contract

5.1 Failure Evidence Input

For each failed test, normalize the following evidence where available:

```
failure_id
test_name
HTTP method + path
operationId
request URL / headers / body
expected status / actual status
response headers / body
OpenAPI request validation errors
OpenAPI response validation errors
application exception / stack trace
application logs
SQLite/database error
test assertion error
relevant OpenAPI operation fragment
```

Missing evidence must remain missing. Never invent evidence. Redact authorization tokens, credentials and secrets before any evidence is sent to an external AI model.

5.2 Required Root-Cause Classes

Classification	Use When
REQUEST_CONTRACT	Request violates the protected OpenAPI contract.
RESPONSE_CONTRACT	Response violates the protected OpenAPI contract.
ROUTING	Route, method, path parameter or endpoint mapping is wrong/missing.
APPLICATION_LOGIC	HTTP/schema plumbing works but application behavior violates the tested rule.
DATABASE_OR_DATA	Failure originates in SQLite state, constraints, fixture data, query or persistence.
APPLICATION_EXCEPTION	Unhandled application exception / 5xx implementation failure.
HARNESS_OR_TEST	The Gateway satisfies the contract but Harness/test logic or expectation is wrong.
ENVIRONMENT_OR_CONFIG	Runtime/config/dependency/startup problem.
SPEC_AMBIGUITY	The protected specification does not uniquely determine expected behavior.
UNKNOWN	Evidence is insufficient for a responsible diagnosis.

Proprietary and Copyright 2026 B. Gopinath

5.3 triage.json Output


```
{
  "failure_id": "F-001",
  "classification": "RESPONSE_CONTRACT",
  "confidence": 0.96,
  "summary": "POST /orders returned a response missing required property 'id'.",
  "first_failing_observation": "OpenAPI response validation failed.",
  "suspected_component": "orders response serialization",
  "spec_reference": "POST /orders -> 201 response",
  "evidence": [
    "HTTP status was 201 as expected",
    "required property 'id' was absent",
    "OpenAPI response validator reported the missing property"
  ],
  "likely_root_cause": "Response construction omitted the created order id.",
  "next_check": "Inspect POST /orders response construction after database insert.",
  "candidate_fix_area": "application",
  "needs_human_review": false
}
```


6. Diagnostic Method

Reason from strongest deterministic evidence to weaker inference. Suggested order:

1. Harness execution failure?
2. Request violates protected OpenAPI?
3. Route/method failure?
4. Application exception?
5. Database error?
6. Expected HTTP status mismatch?
7. Response violates protected OpenAPI?
8. Business/application behavior mismatch?
9. Test expectation appears wrong?
10. Specification ambiguous?
11. Insufficient evidence -> UNKNOWN

Do not ask the LLM to rediscover facts already established by validators. Use deterministic code first; use AI to reason across the remaining evidence.

6.1 AI Role

- Provide only the relevant OpenAPI operation, normalized failure evidence, validator results, relevant logs/excerpts, allowed classifications and required JSON schema.
- Prompt: identify the earliest supported cause in the failure chain; do not confuse downstream symptoms with root cause; cite only supplied evidence; return UNKNOWN when evidence is insufficient and SPEC_AMBIGUITY when the contract does not determine the answer.
- Validate model output as JSON. Retry once with the validation error. If still invalid, emit a machine-readable UNKNOWN / AI-output failure result.
- Keep one small model adapter so the Product is not architecturally tied to one vendor.

7. Acceptance Tests - Objective PASS / FAIL

7.1 Gate A - OpenAPI/Swagger Interoperability

- Target Gateway serves GET /openapi.json.
- Target /openapi.json is semantically identical to the protected canonical OpenAPI 3.1.2 contract.
- Target Gateway serves Swagger UI at GET /docs.
- Swagger UI renders that same /openapi.json and “Try it out” can execute representative acceptance operations.
- No separate documentation/test schema or Gateway-specific mapping is required.

7.2 Gate B - Mandatory Cross-Test Matrix

- A passes only after the same unchanged RCA/Harness tests B and C.
- B passes only after the same unchanged RCA/Harness tests A and C.
- C passes only after the same unchanged RCA/Harness tests A and B.
- Changing code, endpoint mappings, prompts or adapters between target Gateways is a failure.

Proprietary and Copyright 2026 B. Gopinath

- Self-testing may be reported as development evidence but cannot substitute for either required cross-test.

7.3 Gate C - RCA Known-Fault Accuracy

Cross-testing proves interoperability, but a fully correct Gateway might produce few failures. Therefore the RCA must also pass controlled known-fault cases so its diagnostic behavior is objectively exercised:

Injected Fault	Expected Primary Class
Invalid request type/schema	REQUEST_CONTRACT
Required response property omitted	RESPONSE_CONTRACT
Wrong/missing route or method	ROUTING
SQLite constraint/data failure	DATABASE_OR_DATA
Controlled unhandled application exception	APPLICATION_EXCEPTION
Harness expectation contradicts protected contract	HARNESS_OR_TEST
Behavioral/business-rule defect after schema passes	APPLICATION_LOGIC
Evidence intentionally insufficient	UNKNOWN

7.4 Gate D - Non-Tampering / Regression

- RCA does not modify the protected OpenAPI contract, Gateway source, Harness tests or expected results.
- AI cannot make itself pass by weakening the judge.
- Full existing Harness regression still passes after RCA integration.
- Missing/ambiguous evidence results in UNKNOWN or SPEC_AMBIGUITY rather than fabricated certainty.
- Model outage, timeout or invalid output does not crash the Harness.

8. Failure / Recovery / Regression Tests

- LLM API unavailable or times out.
- LLM returns invalid JSON twice.
- Missing application log or stack trace.
- Malformed failure bundle.
- Extremely long log; retain only relevant bounded excerpts.
- Non-JSON or empty API response where the contract expects a body.
- Multiple downstream symptoms produced by one injected root cause.
- Authorization/secret values appear in captured headers and are redacted before model use.

9. AI-First HOW TO - Six-Hour Execution

Time	Mode	Required Outcome
0:00-0:30	Inspect	Give AI this SOW + actual Harness repo + protected OpenAPI + sample PASS/FAIL artifacts. Find existing validators/logging and the smallest RCA integration point. Freeze choices.
0:30-1:20	Normalize evidence	Create one failure-bundle structure and get one real Harness failure -> RCA input end-to-end.
1:20-2:10	Deterministic triage	Implement request/response validation, routing/status, exception, database and Harness execution classifications. Add tests immediately.
2:10-2:50	AI diagnosis	Add tiny model adapter, structured RCA prompt, JSON validation, one retry and UNKNOWN fallback.
2:50-4:05	Cross-test	Test the other two Gateways using only base URL + protected contract. Verify /openapi.json equivalence and Swagger /docs. No target-specific changes.
4:05-4:55	Known faults	Run controlled fault suite; compare injected cause vs reported classification; fix exact failures.
4:55-5:15	Failure/edge	Model outage, missing evidence, ambiguity, redaction and full regression.
5:15-5:30	Scope freeze	No new features. Only acceptance blockers.
5:30-6:00	Review + deliver	AI-review final diff/logs/tests for hard-coding, weakened validation, target-specific adapters, secret leakage and dead code. Rerun; Git push; verify deployment; concise README.

10. Deliverables by Hour 6

- RCA module and small CLI/integration entry point.

Proprietary and Copyright 2026 B. Gopinath

- triage.schema.json and at least one example triage.json.
- Automated known-fault tests.
- Cross-test evidence for both required independent Gateways.
- Protected-contract equivalence results for target /openapi.json.
- Swagger /docs human-test evidence for both target Gateways.
- Successful full Harness regression log.
- Concise README with one standard command for target testing and one for RCA analysis.
- Final Git commit hash and pushed source/config.

10.1 Standard Invocation Contract

Use existing repository commands if already frozen. Otherwise expose one boring, scriptable interface equivalent to:

```
python -m safi_harness test \
  --spec contracts/openapi.yaml \
  --base-url https://gateway-b.example \
  --output results.json

python -m safi_rca analyze \
  --input results.json \
  --output triage.json
```

The target Gateway changes only by --base-url. The code, protected spec and adapter set do not change between Gateway B and Gateway C.

11. Out of Scope / DO NOT BUILD

- Automatic code repair, Git commits or pull requests.
- Test rewriting or OpenAPI rewriting.
- Custom Swagger replacement, dashboard or RCA chat UI.
- Historical RCA database, vector search/RAG or generic log platform.
- NATS, Redpanda or MCP Gateway integration.
- Distributed tracing platform or production observability system.
- Multi-model voting/orchestration framework.
- Restaurant-specific root-cause rules or Gateway-specific adapters.

If any of these appears necessary, stop. It belongs in another SOW.

12. Definition of Done

```
Gateway B + protected OpenAPI -----> unchanged Harness A -----> PASS/FAIL
                                     |
                                     +---- FAIL ----> RCA A ----> triage.json

Gateway C + protected OpenAPI -----> same Harness A ---------> PASS/FAIL
                                     |
                                     +---- FAIL ----> same RCA A -> triage.json
```

SOW-04 is DONE only when the developer can prove all of the following:

- their unchanged RCA/Harness interoperates with both other independently built Gateways;
- both target /openapi.json descriptions match the protected canonical contract;
- both target /docs endpoints provide working Swagger UI human testing from that same contract;
- controlled known-fault cases produce the required root-cause classes;
- the RCA does not repair, weaken or rewrite the judge;
- AI/provider failure is contained and uncertainty is admitted;
- full regressions pass; and
- source/config/tests/evidence are committed and pushed through company Git infrastructure.

THE ACCEPTANCE SENTENCE

If A cannot test B and C directly from the protected OpenAPI contract, A does not pass. If B cannot test A and C, B does not pass. If C cannot test A and B, C does not pass.

13. Six-Hour Rule

Understanding, AI work, coding, integration, Swagger verification, cross-testing of two independent Gateways, known-fault testing, failure testing, fixing, regression, Git delivery and deployment/execution proof must all fit within six working hours maximum.

Proprietary and Copyright 2026 B. Gopinath